High-Level Language

Pre-Exam Review Class



Tsedenbal Javkhlan

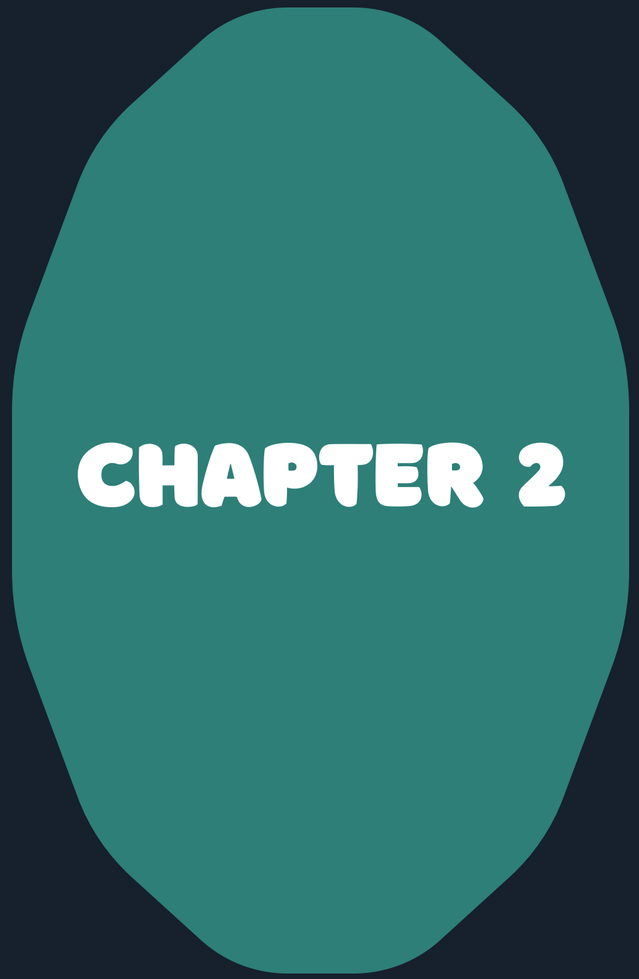
Hello everyone! Today’s session is a pre-exam review for your upcoming test High Level Language Course. We’ll be walking through all the core programming concepts you’ve learned so far, refreshing key definitions, and reviewing essential code structures.

Chapters Covered

- Basic syntax of C++
- Operators and Constants
- Simple IO
- Programming Control Structures
- Arrays and Structures

Basic Syntax of C++ (Chapter 2)

- Variables
- Data types and sizes
- Literals
- Define, initialize, assign



Variable

A variable is just a named container in the computer's memory. The name of the container is called an "identifier".

Example: int age = 20;

Think of a variable as a labeled box. The label is age, and the box holds 20.

Naming rules (Must Knows)

Below are the naming rules you should follow when naming a variable.

Use **letters**, **numbers**, and **underscores** (_).

Start with a **letter**, not a **number**. **score1** is OK, **1score** is not.

No spaces. **my score** is not allowed, but **my_score** is.

Upper and lower case are different. **Age** and **age** are two different variables.

Don’t use C++ words like **int** as names.

Data type

A data type tells the computer what kind of value a variable holds and how much memory it uses.

Because C++ is a statically typed language, you **must write the type** before you use a variable.

Data Types in C++

Basic Data types

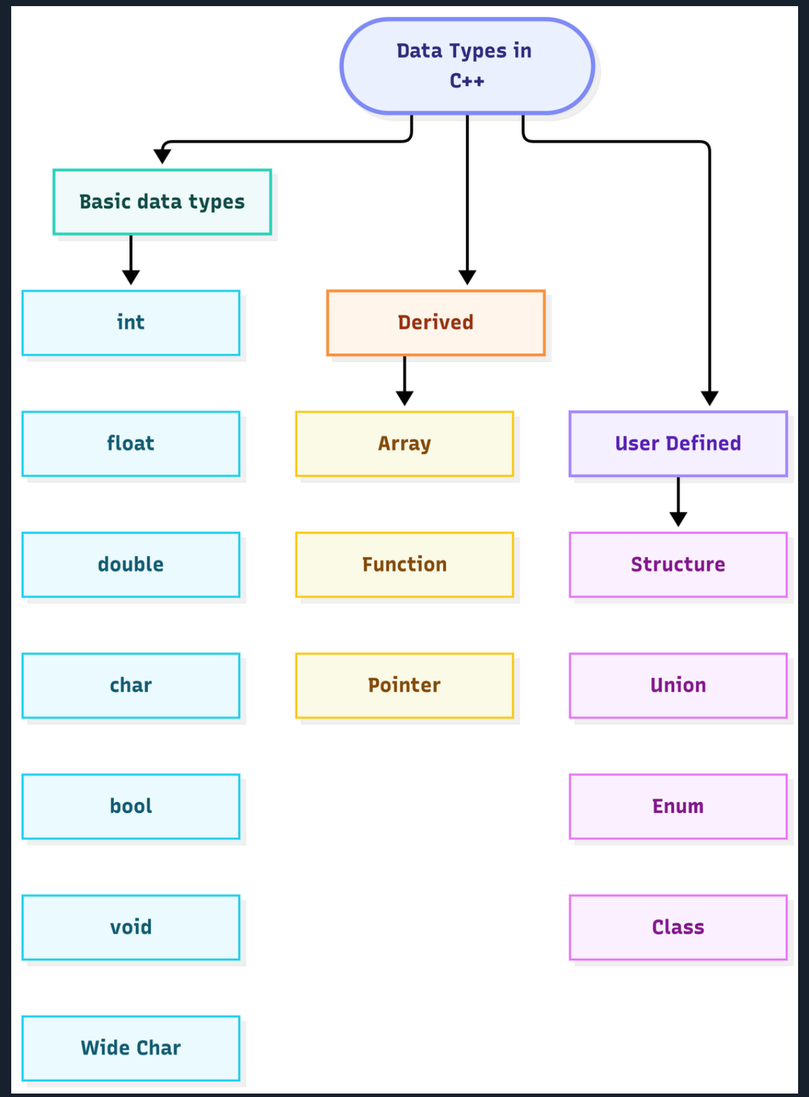
- known as **primitive** or **built-in** data types
- comes **pre-packaged** with a programming language

Derived Data Types

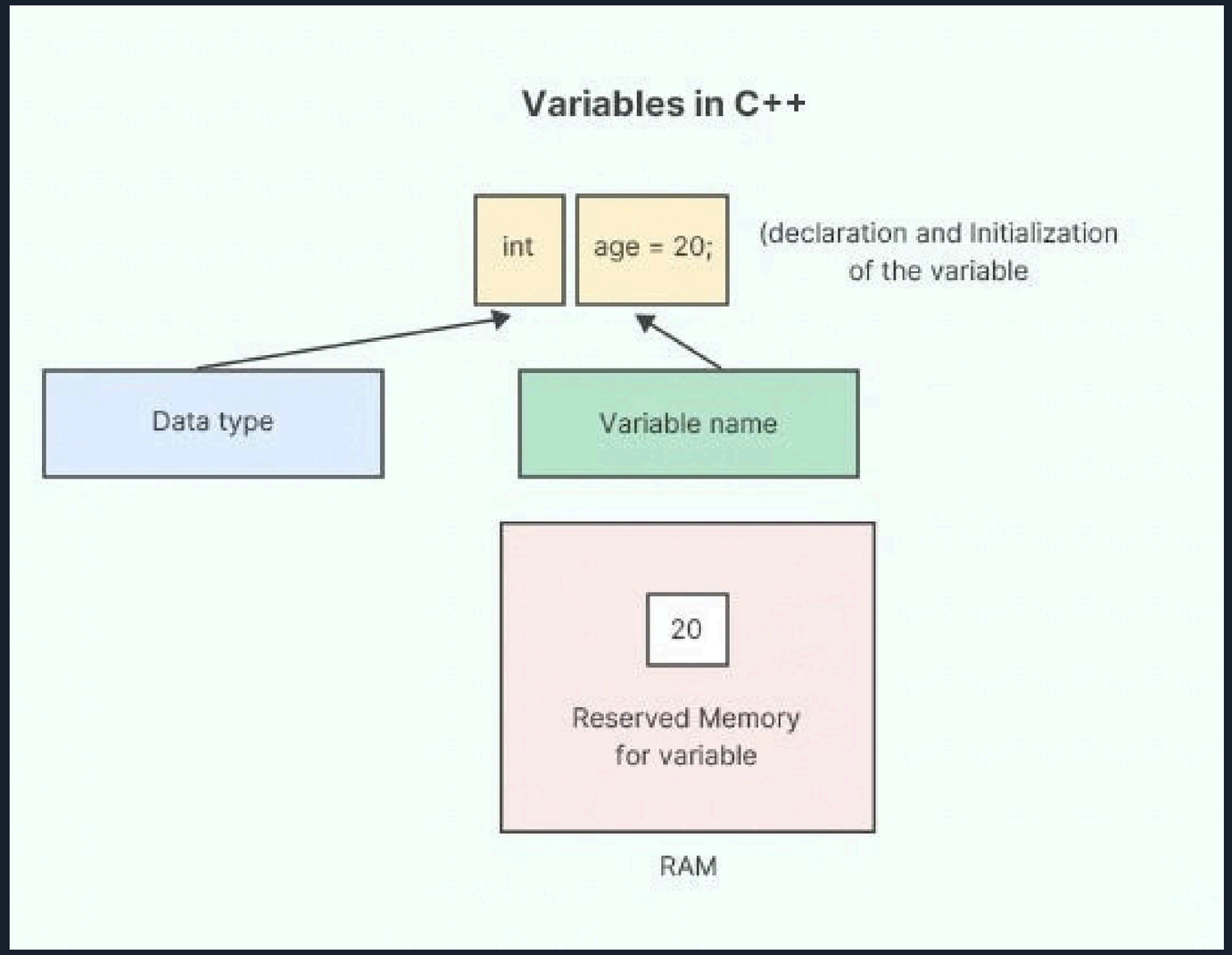
- created by **modifying** or **combining** basic data types together to add more capabilities

User Defined

- completely **custom** designed by the programmer.
- let programmers **group** different data types together to represent real-world objects.



Variable and Data type



int is the type, age is the name, and 20 is the value.

Core Data Types

Type	What it holds	Example
`int`	whole numbers	`20`, `-3`
`double`	decimal numbers	`3.14`
`char`	one character	`'A'`
`bool`	true or false	`true`

1. **int** hold **whole numbers**;
2. **double** hold **decimal numbers**;
3. **char** holds **a character**;
4. **bool** holds **true** (yes, on, 1) or **false** (no, off, 0).

Literals

A literal is just a fixed, raw value typed directly into your code, like the number 5 or the text "hello"

Like, literally.

Define, Initialize, Assign

```cpp
#include <iostream>

int main() {
    // 1. Define
    int myNumber;

    // 2. Define and Initialize
    int score = 100;

    // 3. (Changing the value of an already existing variable)
    myNumber = 42;  // Assigning a value to the defined variable
    score = 150;     // Overwriting the initial value with a new assignment

    return 0;
}
```

Pitfalls

- 'A' (single quotes) is one character. "A" (double quotes) is text. They are not the same.
- **int x = 5.9**; **makes x equal 5**. The decimal part is **cut off**.
- A variable you never give a value **holds a random number**. Always set a value.



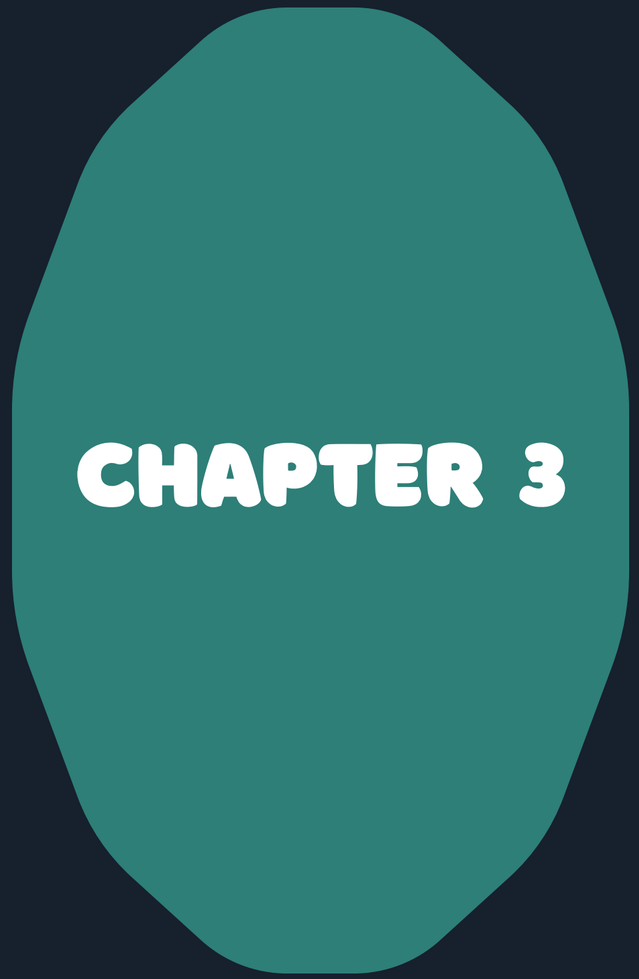
Garbage Value





Operators and Constants

- Operators Overview
- Arithmetic Operators
- Assignment Operators
- Increment & Decrement
- Constants



Operators Overview

An operator is a symbol that does something to values. Example: c = a + b uses + to add a and b.

c = a + b, where + is the operator and a and b are operands.

Operator types

- **Math:** `+ - * / %`
- **Assignment:** `= += -= *= /= %=`
- **Add or subtract 1::** `++ --`
- **Compare:** `== != > < >= <=`
- **Logical:** `&& || !`
- **Member/access and address-of:** `. &`

Arithmetic Operators

+ (addition operator): **adds** two values together. 7 + 3 gives 10.

- (subtraction operator): **subtracts** two values. 7 - 3 gives 4.

* (multiplication operator): **multiplies** two values. 7 * 3 gives 21.

% (modulus operator): **gives the remainder** after dividing. 17 % 5 gives 2.

/ (division operator): **divides** the left value by the right value. **17 / 5 gives 3,** because whole numbers divided by whole numbers keep only the whole part.

Important:

You can't use the modulus operator on float type variables. If you attempt to compile code like 5.5 % 2.0, the compiler will throw an error

Assignment Operators

Must-know

Simple: a = 100 (puts 100 in a)

Multiple: a = b = c = 100 (right to left) **c** gets 100 first, **then b** gets the value of c, **then a** gets the value of b. All three end up as 100.

Compound: a += 5 means a = a + 5

Compound forms: “+=” “–=” “*=” “/=” “%=”

Avoid

No space inside a compound operator: += not + =

Increment and Decrement

- **a++ means “use the value “a” first, then add 1.**
- **++a means “add 1 first, then use the value.**

Examples:

a = 3; b = ++a; gives a = 4, b = 4

a = 3; b = a++; gives a = 4, b = 3

Avoid:

Avoid two or more ++ -- on one line

Pre-increment (++a) increases the value of a first and then uses the new value in the expression

post-increment (a++) uses the current value of a first and increases it afterward.

Operator Precedence

What does each line give?

- 2 + 3 * 4 → 14 (multiply first)
- 10 - 4 - 3 → 3 (left to right)
- (2 + 3) * 4 → 20 (brackets first)

Operators with higher ranking are done first. When two operators have equal ranking, do the one on the left first.

Type	Operator	Precedence Level
Parentheses	( )	High
Unary Operator	! ~ -	
Multiplication/ Division/Modulus	* / %	
Addition/ Subtraction	+ -	
Shift	<< >>	
Comparison	< <= > >=	
Equality	== !=	
Bitwise AND	&	
Bitwise XOR	^	
Bitwise OR	|	
Logical AND	&&	
Logical OR	||	Low

Example

Result (a = 10, b = 3)
13
7
30
3
1

Notice that **10 / 3 gives 3, not 3.33**. Both numbers are whole numbers, so the decimal part is dropped.

```c
int main() {
    int a = 10, b = 3;

    // Addition: Adds two values together.
    int add = a + b; // 13

    // Subtraction: Subtracts the right value from the left value.
    int sub = a - b; // 7

    // Multiplication: Multiplies two values.
    int mul = a * b; // 30

    // Division: Divides the left value by the right value.
    int div = a / b; // 3

    // Modulus: Returns the remainder of integer division.
    int mod = a % b; // 1

    // Increment: Increases a variable's value by 1.
    int x = 5;
    x++; // 6

    // Decrement: Decreases a variable's value by 1.
    int y = 5;
    y--; // 4
```

const Variables

MAIN IDEA

A const is a variable that, after you set its value, nothing in the program can change it. It is same as if you are retarded, and no one can just change it that easily

SYNTAX

```
const double PI = 3.1415926;
```

PRACTICAL USE

```
const int N = 10;
int arr[N];
```

A const value can size an array.

If your code tries to change the const variable, the compiler stops with an error.

You must give a const variable its value when you create it. The line const int x; fails, because the box is locked with nothing inside.

Use const for a number that should **never change**, such as the **size** of an **array**. If you later decide the size should be 20, you change it in one place.

Chapter 3 Summary

Operators are **symbols** that **do math**, **compare values**, or combine yes/no checks.

*** / %** happen before + -. Equal operators go left to right.

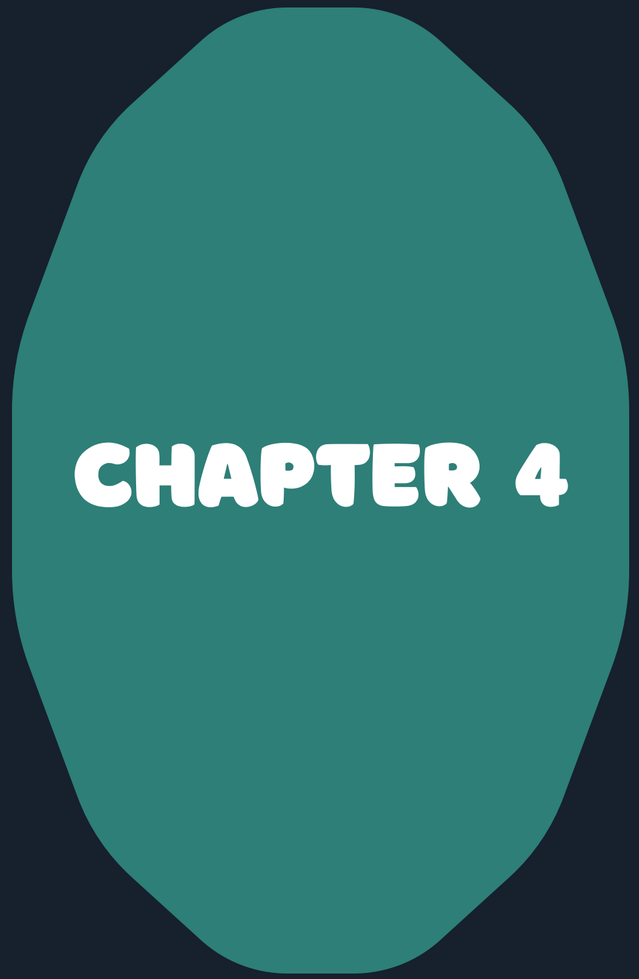
No spaces. **my score** is not allowed, but **my_score** is.

Dividing two whole numbers **drops** the **decimal part**.

A **const** value **cannot change.**

Simple IO

- Print to the screen with cout
- Read from the keyboard with cin
- Format output so it lines up neatly



Input & Output

- A **program talks** to people **through** the **keyboard** and the **screen**.
- **Input** is what the user types in.
- **Output** is what the program shows back on the screen.

C++ calls this flow a “stream.” You don’t need to remember the name.

Data Stream

- **cout** sends things to the **screen**.
- **cin** takes things from the **keyboard**.

cout

- **cout**: cout prints to the screen **using <<**
- **<<**: The insertion operator is used to write data to cout.

Standard Output Stream (cout)

```cpp
// 1. Printing plain text (strings enclosed in double quotes)
cout << "Hello, World!";

// 2. Printing variable values
int age = 20;
cout << age;

// 3. Chaining multiple items together on one line using <<
cout << "I am " << age << " years old." << endl;

// 4. Using '\n' to insert a new line (an alternative to endl)
cout << "First line\nSecond line";
```

Plain text: put words in double quotes

A variable: no quotes

Mixing them: use << between each piece

cin

- **cin**: cin **reads** what the **user types and uses >>** to store it in a variable.

- **>>**: The **extraction operator** is used to read data from cin.

Standard Input Stream (cin)

```cpp
// 1. Reading basic user input into a variable
int age;
cout << "Enter your age: ";
cin >> age;

// 2. Reading multiple values in a single statement
int num1, num2;
cout << "Enter two numbers: ";
cin >> num1 >> num2;
```

```
Enter your age: 18
Enter two numbers: 6 7 _
```

- cin >> age; **waits** for the **user to type a number** and **puts** it in **age**.
- You can read two values in one line: cin >> num1 >> num2;

Manipulators

Manipulators are small instructions you put inside cout to change how things look, like the width of a column or the number of decimal places.

• They are inserted directly into the stream using the **insertion operator (<<)** to modify stream behavior.

• Similar to how “**endl**” ends a line, other manipulators control traits like spacing or precision.

• Used when you need to **format output**, such as setting decimal precision for floating-point numbers or aligning columns.

• Remember that most stream manipulators require including the **<iomanip> library header**.



setw()

1. It sets the **minimum width** (number of character spaces) for the next output item. It helps align text and numbers into neat columns

2. It only affects the **very next** output value, so you **must call setw()** before every single item you want to pad.

3. For example **setw(10) makes the next item take up 10 spaces**. Use it to line up columns.

Code Example

```cpp
#include <iostream>
#include <iomanip> // Required for setw

int main() {
    // 1. Basic formatting (Default right-alignment)
    std::cout << std::setw(10) << "Apple" << std::setw(10) << 5 << "\n";
    std::cout << std::setw(10) << "Banana" << std::setw(10) << 12 << "\n";

    // 2. Combining with std::left to make text left-aligned
    std::cout << "\nLeft-Aligned Column:\n";
    std::cout << std::left << std::setw(15) << "Item Name" << "Price\n";
    std::cout << std::left << std::setw(15) << "Laptop" << "$999\n";

    return 0;
}
```

output



```
     Apple          5
    Banana         12

Left-Aligned Column:
Item Name      Price
Laptop         $999
```

setfill()

1. **Changes** the default **blank** padding character (used by setw) to a **different character**, such as a zero (0) or an asterisk (*).

2. Once you set a fill character, it stays **active** for all future setw outputs **until** you explicitly change it.

3. For example **setfill('0') fills** the **empty spaces with** a character, like **0.** Good for numbers like 0042.

Code Example

```cpp
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    cout << setfill('0') << setw(4) << 7 << endl;
    cout << setfill('*') << setw(5) << 99 << endl;
    return 0;
}
```

output



```
0007
***99
```

setprecision()

1. setprecision() **controls** how many **digits** a **decimal number shows**.

2. By default, it **counts** total **significant digits** rather than decimal places, unless it is combined with **fixed**.

3. Add **fixed** to show the **decimal places**: fixed << setprecision(2) shows 3.14.

Code Example

```cpp
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    double pi = 3.14159;
    cout << fixed << setprecision(2) << pi << endl;
    return 0;
}
```

output



```
3.14
```

Single-character I/O

Single-character I/O (Input/Output) means reading or writing data one single character (or byte) at a time from a stream or device, such as a keyboard or file, without complex formatting.

- Use cin.get(ch) to read one character, including spaces.

- Use cout.put(ch) to print one character.



cin.get(ch)

1. Reads a **single character** from the input stream, **including** whitespace characters, without skipping them.

2. Basically **cin.get(ch)** reads one character. It reads spaces too.

3. Used when you need to **read every single keystroke** or space a **user types**. For example, if you want your program to pause and **wait** for the **user** to **press** the **spacebar or** enter a **blank line**, **cin.get(ch) captures** it **instead** of **ignoring** it.

Code Example

```cpp
#include <iostream>
using namespace std;
int main() {
    char ch;
    cout << "Type a character: ";
    cin.get(ch);
    cout << "You typed it successfully!" << endl;
    return 0;
}
```

output



```
Type a character: X
You typed it successfully!
```

cout.put(ch)

1. Basically cout.put(ch) prints one character.

2. It **only accepts** character data or character variables, **not** standard **text strings** in double quotes , for example “word”, “hello” are not accepted.

Code Example

```cpp
#include <iostream>
using namespace std;
int main() {
    char ch = 'K';
    cout.put(ch);
    cout << endl;
    return 0;
}
```

output



```
K
```

Chapter 4 Basic Summary

cout << shows things on the screen.

cin >> reads what the user types.

setw() sets the **width** of the next item.

setprecision() sets how many decimal digits to show.

cin.get reads one character, including spaces.

Quick Check

cout << setw(4) << 7; prints ?

(three spaces, then 7)

cout << fixed << setprecision(1) << 2.25; prints ?

Prints: 2.3

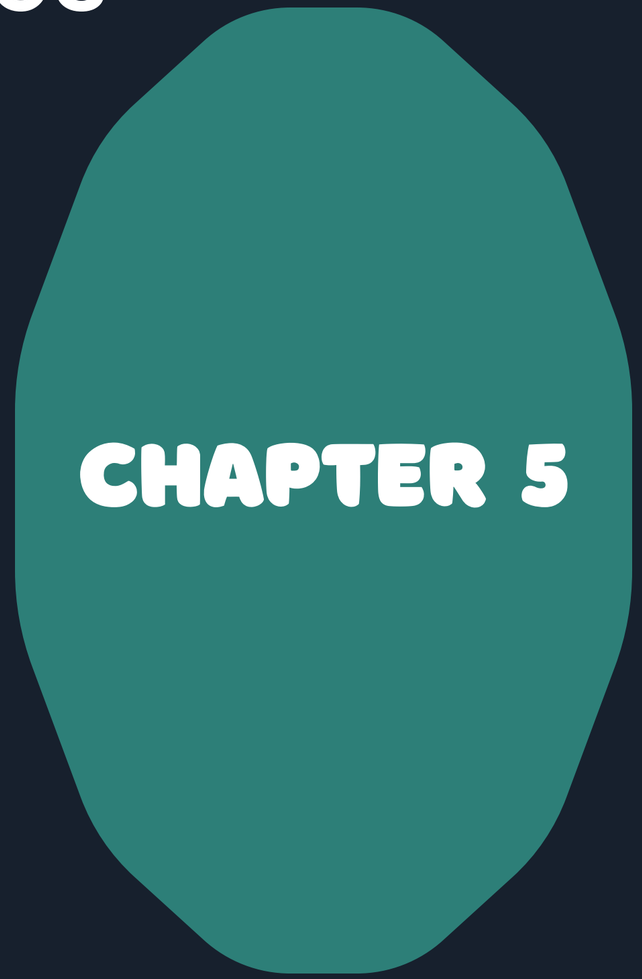
The user types **John Smith,** and **cin >> name; stores ?**

John

The **extraction operator (>>)** treats spaces, tabs, and newlines as **separators**.

Programming Control Structures

- Relational operators
- Selection statements
- Logical operators
- Iteration (loops)



Relational operators

1. >, <, >=, <=, ==, != compare two values or expressions and return 1 (true) or 0 (false).

2. They have lower precedence than arithmetic, so 2 <= 4 - 3 is 2 <= 1, which is 0.

3. Comparisons work across types: int 5 == float 5 gives 1, and bool false == int 0 gives 1.

4. Chaining doesn't work as in math. 1 < b < 3 evaluates (1 < b) first, giving 0 or 1, and both are less than 3, so the result is always 1. Use b > 1 && b < 3 instead.

if statement

An if statement is basically a decision-maker for your code. It allows your program to make choices and run certain lines of code only when a specific condition is met —just like you do in real life ("If I feel lazy, I will skip class").

Code Example

```cpp
#include <iostream>
using namespace std;

int main() {
    int score = 85;

    if (score >= 60) {
        cout << "You passed the exam!" << endl;
    }

    return 0;
}
```

output



```
You passed the exam!
```

Single / Double Selection

Selection is just making choices in code, usually with an if statement.

• Single selection uses an **if statement** to run code only when true, whereas double selection uses an **if-else statement** to handle both true and false paths.

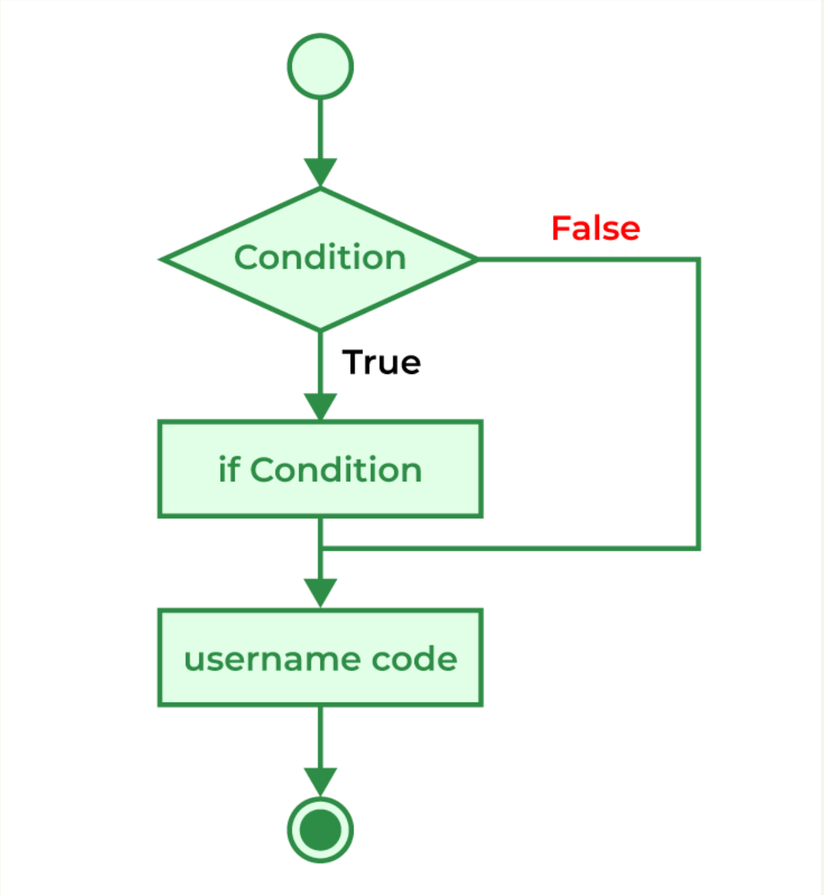
• The else block is **always optional** and automatically binds to the closest preceding if statement.



Single Selection (if)

1. Evaluates a condition; **if** the condition is **true**, **it executes** the enclosed block of code. **If false**, it **skips** that block entirely.

2. It's used for **checking** a specific requirement, for example **applying** a discount **only if** a customer's total exceeds a certain amount.



Code Example

```cpp
#include <iostream>
using namespace std;
int main() {
    int score = 85;
    if (score >= 60) {
        cout << "You passed!" << endl;
    }
    return 0;
}
```

output

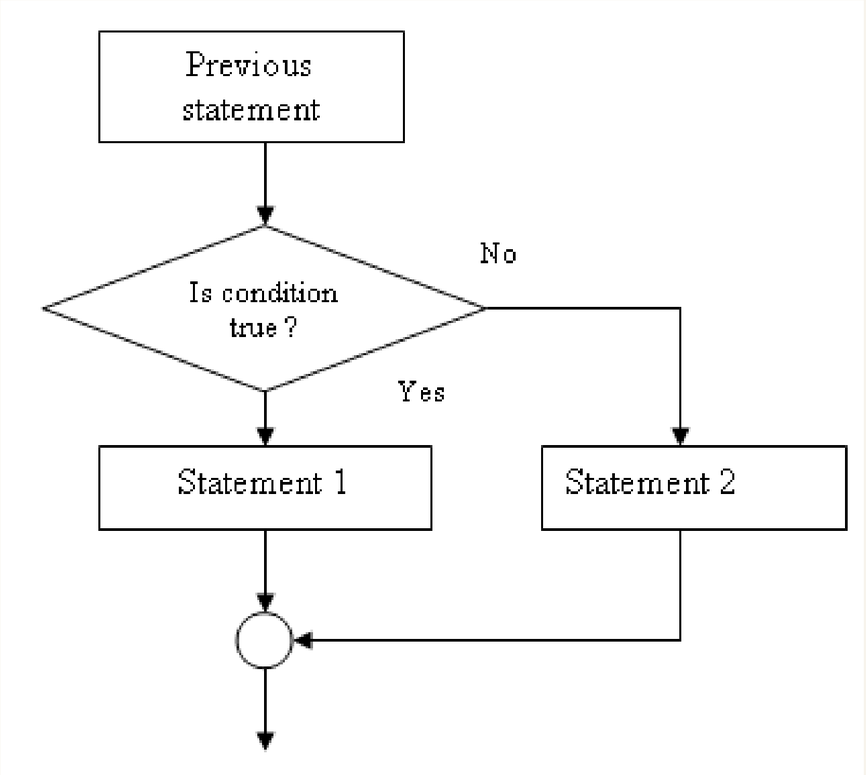


```
You passed!
```

Double Selection (if-else)

1. It's an "**either/or**" switch—it runs one block of code if your condition is true, or the else block if it's false.

2. The **program** will **only** ever **run one** of those **paths**; it can never run both at the same time.



Code Example

```cpp
#include <iostream>
using namespace std;
int main() {
    int number = 7;
    if (number % 2 == 0) {
        cout << "Even number" << endl;
    } else {
        cout << "Odd number" << endl;
    }
    return 0;
}
```

output

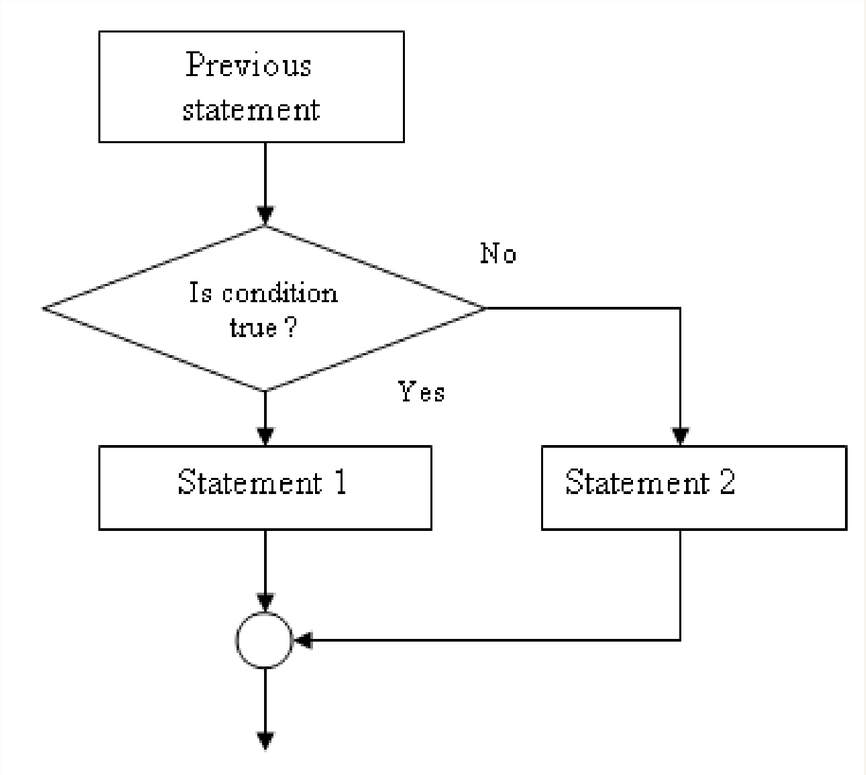


```
Odd number
```

Multi-Selection (if-else if-else)

1. The computer **checks** each option from **top to bottom** and **runs** the very first one **that matches**, ignoring all the rest.

2. It's used for situations with **three or more choices**, like turning a test score into a letter grade (A, B, C, D, or F).



Code Example

```cpp
#include <iostream>
using namespace std;
int main() {
    int score = 75;
    if (score >= 90) {
        cout << "Grade: A" << endl;
    } else if (score >= 80) {
        cout << "Grade: B" << endl;
    } else if (score >= 70) {
        cout << "Grade: C" << endl;
    } else {
        cout << "Grade: F" << endl;
    }
    return 0;
}
```

output

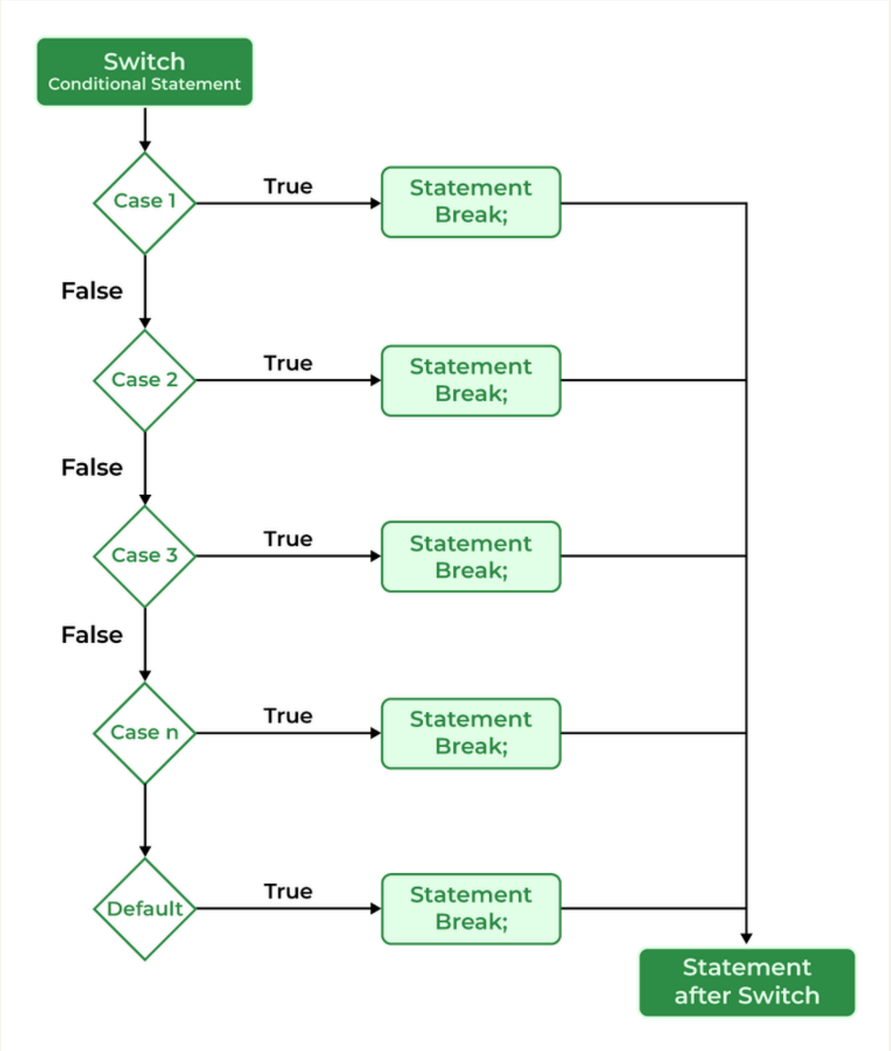


```
Grade: C
```

Switch Statement

1. Compares **a single variable** against a list of exact values (**cases**) and **runs** the **matching block** of code.

2. **Always** put **break; at** the **end** of every case, or the program will accidentally run the code for all the options below it.



Code Example

```cpp
#include <iostream>
using namespace std;
int main() {
    int choice = 2;
    switch(choice) {
        case 1: cout << "Start"; break;
        case 2: cout << "Settings"; break;
        default: cout << "Invalid choice";
    }
    return 0;
}
```

Because **choice** is set to 2, the switch statement scans the options to find a matching case, jumps directly to case 2 to print "Settings", and then uses the break statement to exit the block so it doesn't accidentally run anything else.

output



```
Settings
```

Logical Operators

1. Use && (AND), || (OR), and ! (NOT) to check multiple things at once—like making sure a year is between 1990 and 2000 (year >= 1990 && year < 2000).

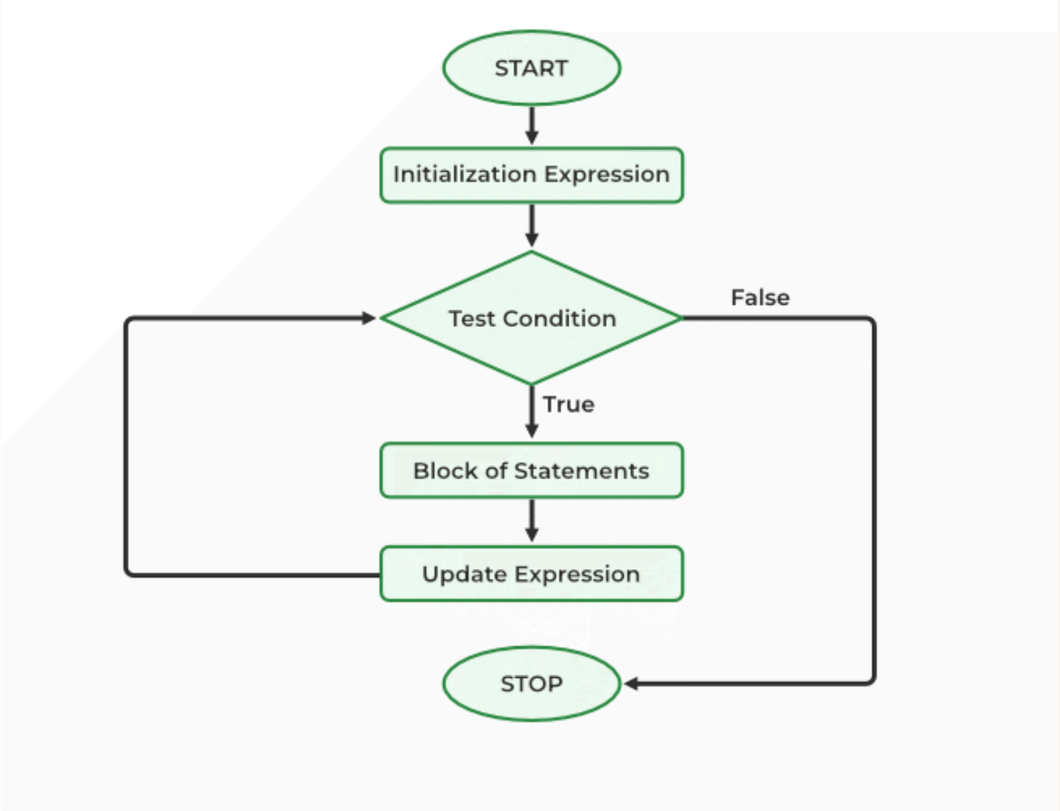
2. Just like math, C++ evaluates conditions in a strict order: ! (exclamation) happens first, then comparisons (>, <), then equality, followed by &&, and || last.

3. If a=3, b=2, c=1, writing a > b > c doesn't work the way you'd think (it evaluates to 0), so you must use && to chain comparisons properly (a > b && b > c), which correctly gives 1.

For loop

1. **Repeats** a block of **code** a specific number of times **using a built-in counter.**

2. The loop header has three parts separated by semicolons: **initialization** (starting point), **condition** (when to stop), and **update** (how the counter changes each time).



Code Example

```cpp
#include <iostream>
using namespace std;
int main() {
    for (int i = 1; i <= 3; i++) {
        cout << "Count: " << i << endl;
    }
    return 0;
}
```

This program uses a loop starting at **int i = 1** that runs as long as **i <= 3**, adding 1 to i after each cycle (**i++**). On every run, the cout command prints "Count:" followed by the current value of i, which is why the output displays the numbers 1, 2, and 3 sequentially.

output

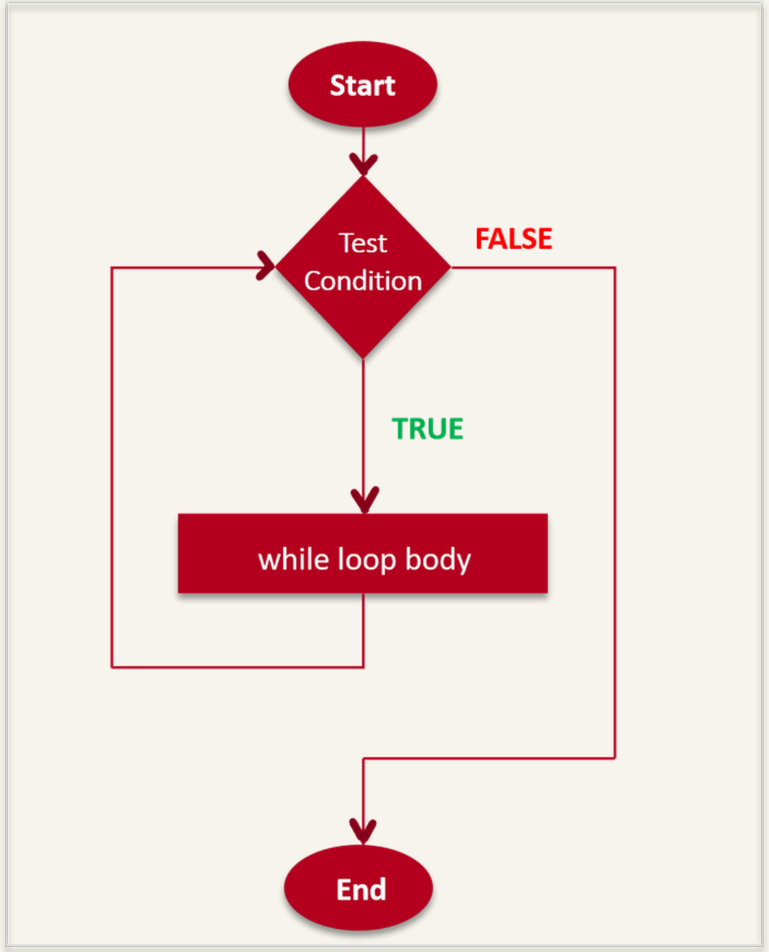


```
Count: 1
Count: 2
Count: 3
```

While Loop

1. **Repeats** a block of code **as long as** a given **condition** remains **true**.

2. For example it’s used **when you don't know ahead of time how many times the loop will run**, such as waiting for a user to type a specific exit command or correct password.



Code Example

```cpp
#include <iostream>
using namespace std;
int main() {
    int count = 1;
    while (count <= 3) {
        cout << "Number: " << count << endl;
        count++;
    }
    return 0;
}
```

This program uses a while loop **starting at int count = 1** that keeps running as long as **count <= 3.** Inside the loop, the cout command prints "Number:" followed by the current value of count, and count++ increases its value by 1 after each cycle, causing the program to print the sequence 1, 2, and 3.

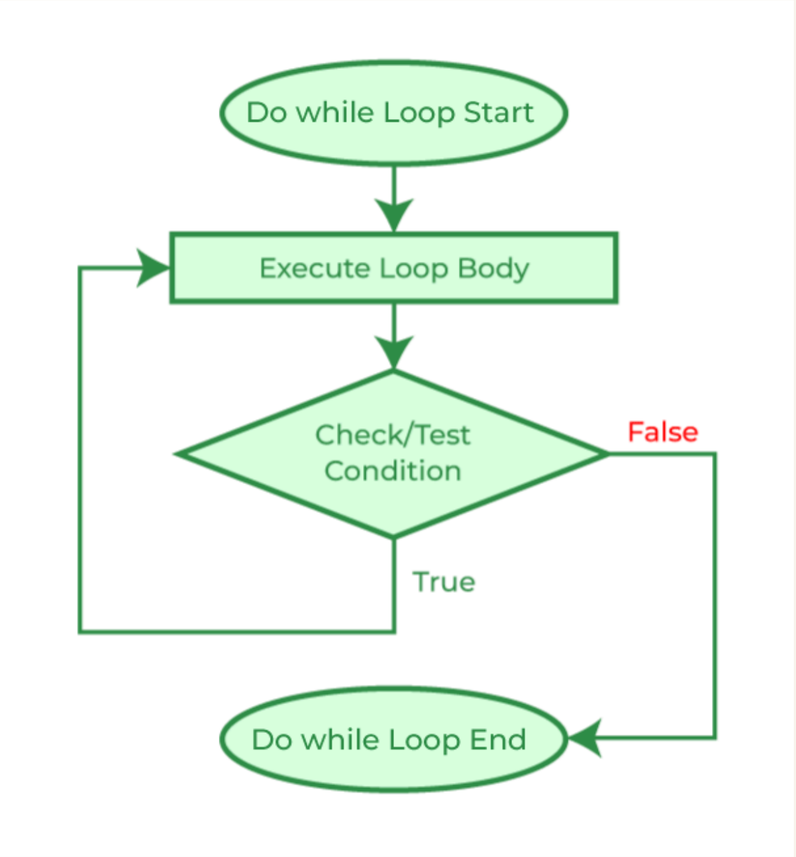
output



```
Number: 1
Number: 2
Number: 3
```

Do-While Loop

1. **Repeats** a block of code **at least once**, then continues repeating as long as the condition stays true.
2. Unlike a regular while loop, the **condition is checked at the end of the loop**, which means the **code block** is guaranteed to **run at least one** time.



Code Example

This program starts at **num = 1** and runs the loop at least once before **checking** the condition **num <= 3**, adding 1 (num++) each time until it finishes.

```cpp
#include <iostream>
using namespace std;
int main() {
    int num = 1;
    do {
        cout << "Value: " << num << endl;
        num++;
    } while (num <= 3);
    return 0;
}
```

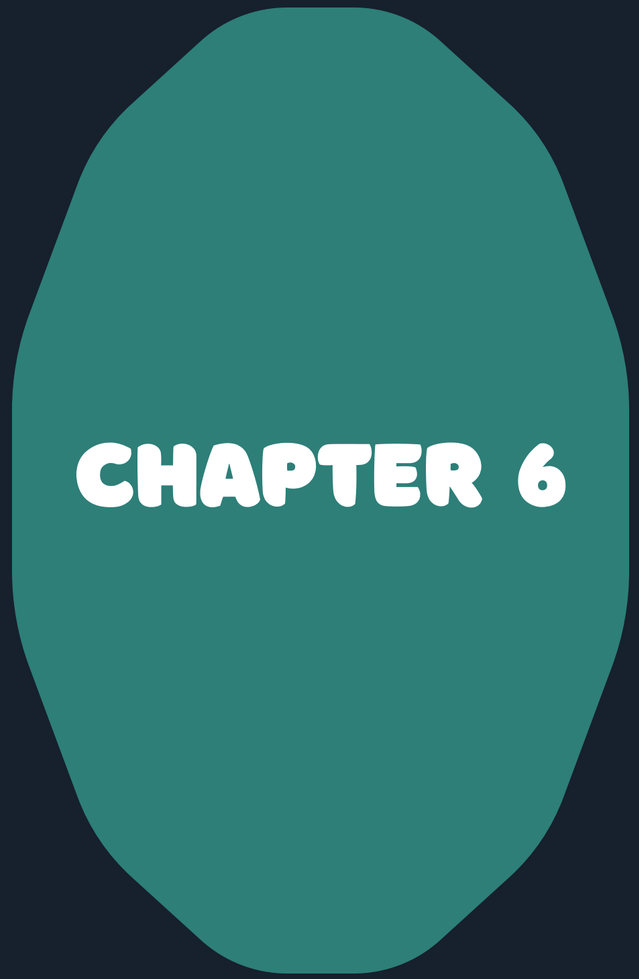
output



```
Value: 1
Value: 2
Value: 3
```

Arrays and Structures

- Arrays
- Structures
- Enumerated data types
- Sort & find



Arrays

1. An array has a fixed size set when you create it, meaning you cannot make it larger or smaller while your program is running.
2. Array positions always start counting at 0 instead of 1, so the first item is at index 0 and the last item is at size - 1.
3. All elements stored inside an array must be of the exact same data type (for example, all int or all double).
4. C++ does not automatically check if your index is valid, so trying to access an index outside your array size can crash your program or corrupt data.

Code Example

This program creates **an array named scores** that stores **three** values. Because arrays start counting at 0, **scores[0] accesses the first item (90)** and scores[2] accesses the last item (95), which are then printed to the screen.

```cpp
#include <iostream>
using namespace std;
int main() {
    int scores[3] = {90, 85, 95};
    cout << "First score: " << scores[0] << endl;
    cout << "Last score: " << scores[2] << endl;
    return 0;
}
```

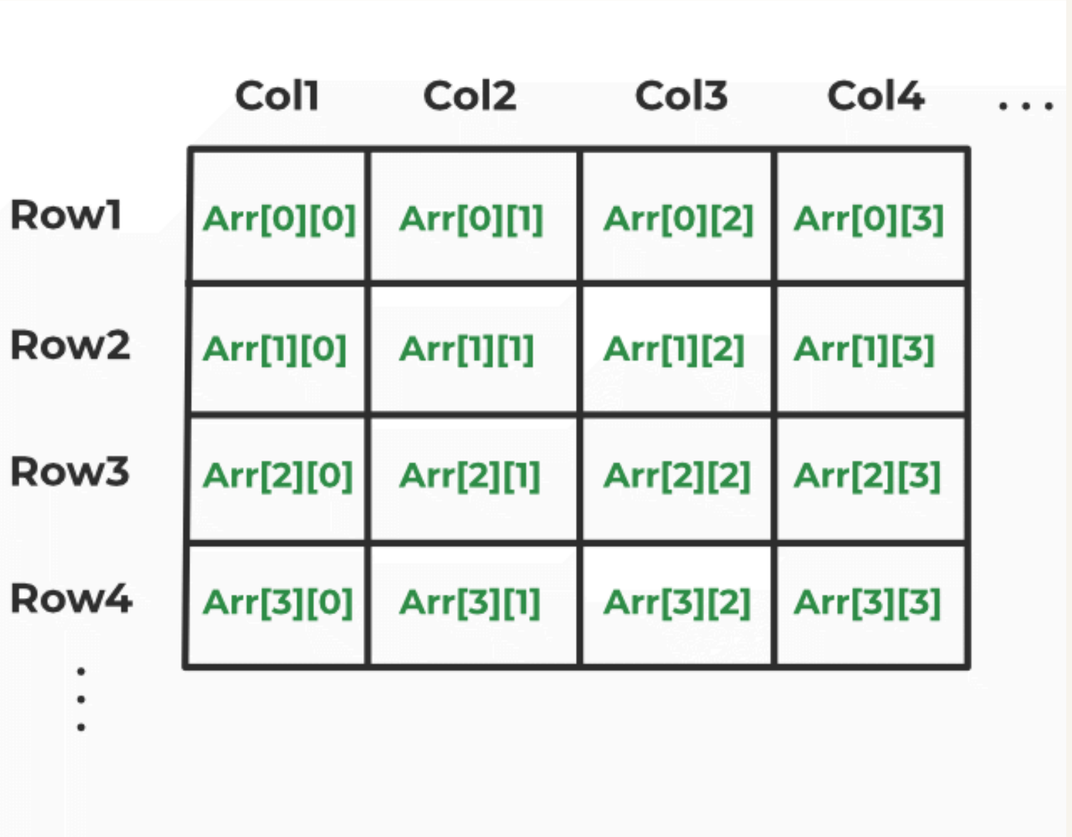
output



```
First score: 90
Last score: 95
```

2D Arrays

1. **Stores data** in a **grid or table format** made up of rows and columns, similar to a spreadsheet or matrix.
2. You **must specify two indexes** when accessing elements: the **first** index represents the **row**, and the **second** index represents the **column** (both starting at 0).



Code Example

This program creates a **2D array called grid with 2 rows and 2 columns**. Because indexing starts at 0, grid[0][1] accesses row 0, column 1 (the value 2) and grid[1][0] accesses row 1, column 0 (the value 3), which are then printed to the screen.

```cpp
#include <iostream>
using namespace std;
int main() {
    int grid[2][2] = {
        {1, 2},
        {3, 4}
    };
    cout << "Row 0, Col 1: " << grid[0][1] << endl;
    cout << "Row 1, Col 0: " << grid[1][0] << endl;
    return 0;
}
```

output



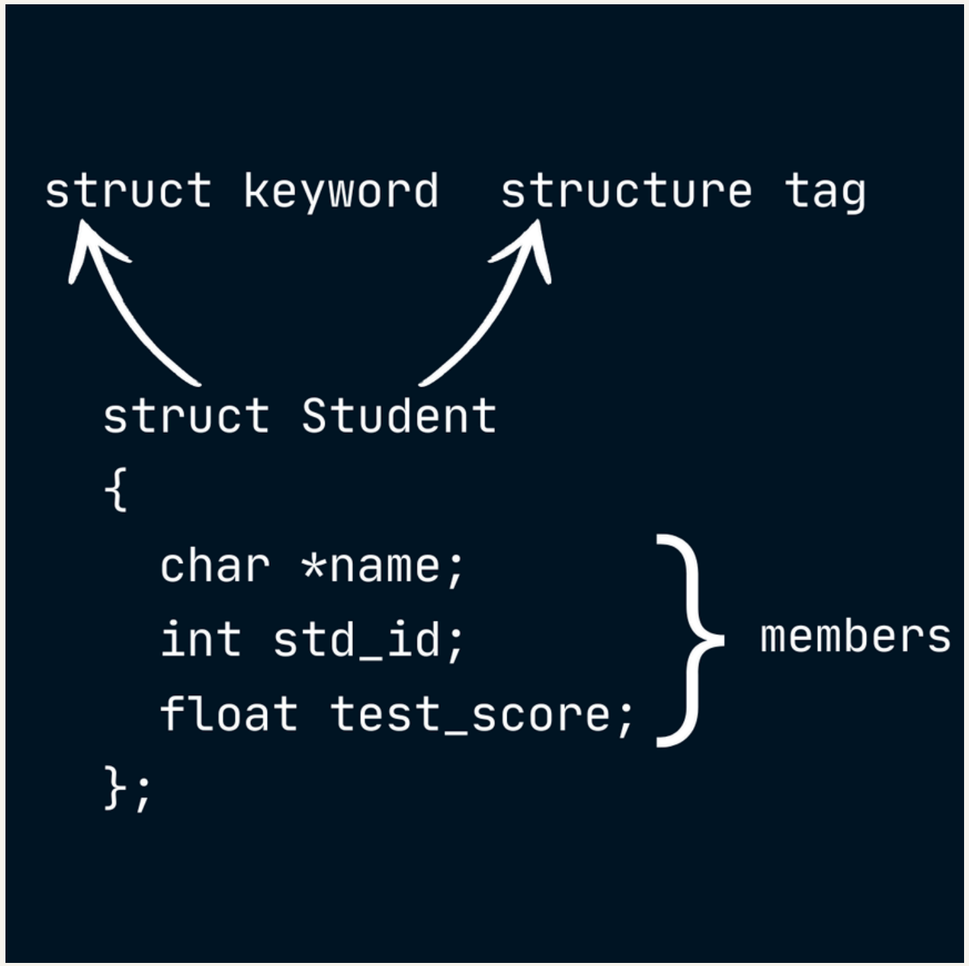
```
Row 0, Col 1: 2
Row 1, Col 0: 3
```

Structures (struct)

1. Groups different variables (potentially of different data types) together under a single custom name to represent a single real-world entity.

2. Use the dot operator (.) to access or modify individual members inside a structure variable.

3. For example, **a Cat struct** can group a **name**, **age**, and **favoriteToy** together so you don't have to keep track of them separately!



Code Example

This code **uses a struct to bundle variables** into one **Cat** package. It creates myCat with its starting values ("Whiskers", 9, true), making it easy to group and access related data.

```cpp
struct Cat {
    string name;
    int livesRemaining;
    bool isPlottingMurder;
};

int main() {
    Cat myCat = {"Whiskers", 9, true};

    cout << "Cat Name: " << myCat.name << endl;
    cout << "Lives Left: " << myCat.livesRemaining << endl;
    cout << "Plotting Murder: " << (myCat.isPlottingMurder ? "Yes" : "No") << endl;

    return 0;
}
```

output 👇

```
Cat Name: Whiskers
Lives Left: 9
Plotting Murder: Yes
```

Enumerated Types (enum)

1. **Creates** a **custom data type** with a set of u**ser-defined** named integer constants.

2. By default, enum items are **automatically assigned integer values** starting from 0 (e.g., Monday is 0, Tuesday is 1, etc.), **unless** you explicitly assign custom numbers.

3. It is used for **making code easier to read** and maintain by **replacing confusing numbers with** meaningful **words** (such as days of the week, compass directions, or menu states).

Enum Items	Value
enum_item1	0
enum_item2	1
enum_item3	2

Code Example

This program uses an **enum** to **create** a custom list of named constants (SUN through WED) where **each option automatically gets a number starting at 0.** It sets today to MON (which equals 1) and prints out its index value.

```cpp
#include <iostream>
using namespace std;
int main() {
    enum Day { SUN, MON, TUE, WED };
    Day today = MON;
    cout << "Day index: " << today << endl;
    return 0;
}
```

output



```
Day index: 1
```

Practice Question

What happens if you forget to include a break; statement inside a case of a switch block?

A The program will crash with a compilation error.

B The program will automatically skip the remaining cases and exit the switch.

C The program will execute the matching case and continue running into subsequent cases (fall-through).

D The compiler will treat the case as a syntax error.

CORRECT ANSWER
iiiisssssss?????
?????????

Practice Question

What happens if you forget to include a break; statement inside a case of a switch block?

A The program will crash with a compilation error.

B The program will automatically skip the remaining cases and exit the switch.

C The program will execute the matching case and continue running into subsequent cases (fall-through).

D The compiler will treat the case as a syntax error.

CORRECT ANSWER is (C)

Practice Question

Which of the following is the most common cause of an infinite while loop?

A Starting the counter variable at 0 instead of 1.

B Forgetting to update or change the loop control variable inside the loop body so the condition never becomes false.

C Placing a semicolon directly after the while condition header.

D Using an integer instead of a boolean type for the condition.

CORRECT ANSWER
iiiisssssss?????
?????????

Practice Question

Which of the following is the most common cause of an infinite while loop?

A Starting the counter variable at 0 instead of 1.

B Forgetting to update or change the loop control variable inside the loop body so the condition never becomes false.

C Placing a semicolon directly after the while condition header.

D Using an integer instead of a boolean type for the condition.

CORRECT ANSWER is (B)

Practice Question

When working with instances of a custom struct, which operator must be used to access individual members (like variables inside the struct)?

A The arrow operator (->)

B The scope resolution operator (::)

C The dot operator (.)

D The dereference operator (*)

CORRECT ANSWER
iiiisssssss?????
?????????

Practice Question

When working with instances of a custom struct, which operator must be used to access individual members (like variables inside the struct)?

A The arrow operator (->)

B The scope resolution operator (::)

C The dot operator (.)

D The dereference operator (*)

CORRECT ANSWER
is (C)

Practice Question

What is the output of the following code?

A) No 5

B) Yes 10

C) Yes 5

D) Compilation Error

```
#include <iostream>
using namespace std;
int main() {
    int x = 5;
    if (x = 10) {
        cout << "Yes ";
    } else {
        cout << "No ";
    }
    cout << x;
    return 0;
}
```

Practice Question

What is the output of the following code?

```
#include <iostream>
using namespace std;
int main() {
    int x = 5;
    if (x = 10) {
        cout << "Yes ";
    } else {
        cout << "No ";
    }
    cout << x;
    return 0;
}
```

B) Yes 10

Practice Question

What is the output of the following code?

A) 2

B) 3

C) 67

D) 9

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[2][3] = {
        {2, 5, 8},
        {1, 6, 9}
    };
    int count = 0;

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            if (arr[i][j] > 5) {
                count++;
            }
        }
    }

    cout << count;
    return 0;
}
```

Practice Question

What is the output of the following code?

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[2][3] = {
        {2, 5, 8},
        {1, 6, 9}
    };
    int count = 0;

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            if (arr[i][j] > 5) {
                count++;
            }
        }
    }

    cout << count;
    return 0;
}
```

B) 3

Definetily not 67

Practice Question

What is the output of the following code?

A) 0

B) 13

C) 14

D) 76

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Rectangle {
    int width;
    int height;
};

int main() {
    Rectangle rect = {4, 5};

    if (rect.width > rect.height) {
        rect.width = rect.width * 2;
    } else {
        rect.height = rect.height * 2;
    }

    cout << rect.width + rect.height;
    return 0;
}
```

Practice Question

What is the output of the following code?

C) 14

```
#include <iostream>
#include <string>
using namespace std;

struct Rectangle {
    int width;
    int height;
};

int main() {
    Rectangle rect = {4, 5};

    if (rect.width > rect.height) {
        rect.width = rect.width * 2;
    } else {
        rect.height = rect.height * 2;
    }

    cout << rect.width + rect.height;
    return 0;
}
```

Practice Question

What is the output of the following code?

A) 6 11 1

B) 6 10 0

C) 5 11 1

D) 6 11 0

```
#include <iostream>
using namespace std;
int main() {
    int x = 5, y = 10;
    int z = (x++ > 5) && (++y > 10);
    cout << x << " " << y << " " << z;
    return 0;
}
```

Practice Question

What is the output of the following code?

B) 6 10 0

```
#include <iostream>
using namespace std;
int main() {
    int x = 5, y = 10;
    int z = (x++ > 5) && (++y > 10);
    cout << x << " " << y << " " << z;
    return 0;
}
```

Practice Question

What is the output of the following code?

A) 29

B) 22

C) 36

D) 15

```
int main() {
    int level = 1;
    int points = 0;
    switch (level) {
        case 1:
            points += 10;
        case 2:
            points += 5;
            break;
        case 3:
            points += 20;
            break;
    }
    while (points < 30) {
        points += 7;
    }
    cout << points;
    return 0;
```

Practice Question

What is the output of the following code?

C) 36

Level is 1, so the switch starts at case 1 and adds 10 to points. There is no break after it, so the code keeps going and also runs case 2, adding 5. Now points is 15, and the break stops the switch.
Next, the while loop runs while points is less than 30. It adds 7 each time: 15 → 22 → 29 → 36. At 29, the condition is still true, so it adds 7 again. The loop stops at 36.

```
int main() {
    int level = 1;
    int points = 0;
    switch (level) {
        case 1:
            points += 10;
        case 2:
            points += 5;
            break;
        case 3:
            points += 20;
            break;
    }
    while (points < 30) {
        points += 7;
    }
    cout << points;
    return 0;
```

Practice Question

What is the output of the following code?

A) 50 4

B) 100 4

C) 70 4

D) 50 0

```
#include <iostream>
using namespace std;

struct Student {
    int score;
    int bonus;
};

int main() {
    Student s = {50, 0};
    for (int i = 1; i <= 4; i++) {
        s.score += i * 5;
        s.bonus = s.bonus + 1;
    }
    cout << s.score << " " << s.bonus;
    return 0;
}
```

Practice Question

What is the output of the following code?

B) 100 4

The loop runs 4 times, with i as 1, 2, 3, 4. Each time, score increases by i * 5: 5 + 10 + 15 + 20 = 50, so the score goes from 50 to 100. bonus goes up by 1 each time, so it ends at 4.

```
#include <iostream>
using namespace std;

struct Student {
    int score;
    int bonus;
};

int main() {
    Student s = {50, 0};
    for (int i = 1; i <= 4; i++) {
        s.score += i * 5;
        s.bonus = s.bonus + 1;
    }
    cout << s.score << " " << s.bonus;
    return 0;
}
```

Thank you for participating in HSIYC AcademiaSphere's High-Level Language Programming Review Class #1!

Your feedback helps us evaluate the effectiveness of our review sessions and improve future academic support activities. This survey takes approximately 1-2 minutes to complete. We appreciate your honest responses!



Thank You